

Agenda (10 min)

1. Problema y motivación (1:00)
2. Objetivos (0:40)
3. Metodología (1:10)
4. Arquitectura y stack (1:20)
5. Datos, experimento y métricas (1:10)
6. Resultados (2:00)
7. Análisis inferencial y validez (1:00)
8. Conclusiones (0:40)
9. Aportes (0:40)
10. Futuro y cierre (0:20)

Notas: - 1 idea por diapositiva; máximo 5 bullets.

- Apóyate en figuras (mejor que texto).

Problema y motivación

- ML debe evaluarse por precisión y por eficiencia/sostenibilidad.
- Escasa evidencia comparativa SMA vs. arquitectura centralizada con energía.
- Impacto: decisiones técnicas y ambientales más informadas.

Fuente: docs/Macroproceso.txt (Intro/Justificación)

Objetivo general

- Diseñar e implementar un prototipo SMA para evaluar eficiencia computacional y sostenibilidad energética en entrenamiento de ML supervisado.
- Entornos simulados, controlados y replicables.

Fuente: docs/Macroproceso.txt §4

Objetivos específicos (resumen)

- OE2: Implementar SMA con RF y SVM.
- OE3: Evaluar eficiencia y sostenibilidad (tiempo, CPU, memoria, energía normalizada).
- OE4: Analizar e interpretar resultados e implicaciones.

Fuente: docs/Macroproceso.txt §5;

docs/achievements/objetivos_especificos.{md,tex}

Metodología (cómo / por qué / para qué)

- Cómo: comparación control (centralizado) vs. tratamiento (SMA); $n \approx 10$ por grupo; análisis por permutación (≥ 5000).
- Por qué: medir diferencias rigurosas con trazabilidad y reproducibilidad.
- Para qué: validar utilidad del SMA y sentar base experimental escalable.

Fuente: docs/achievements/metodologia.{md,tex}

Arquitectura y stack

- SMA en JADE (Java 21/Maven) + Backend FastAPI (métricas estilo Prometheus).
- Observabilidad: Prometheus + Grafana (dashboards precargados).
- Componentes: PREPROCESS → TRAIN → EVAL; orquestación por agentes.

Fuente: `docs/architecture/*` , `docs/bridge_jade.md` ,
`backend/api/main.py`

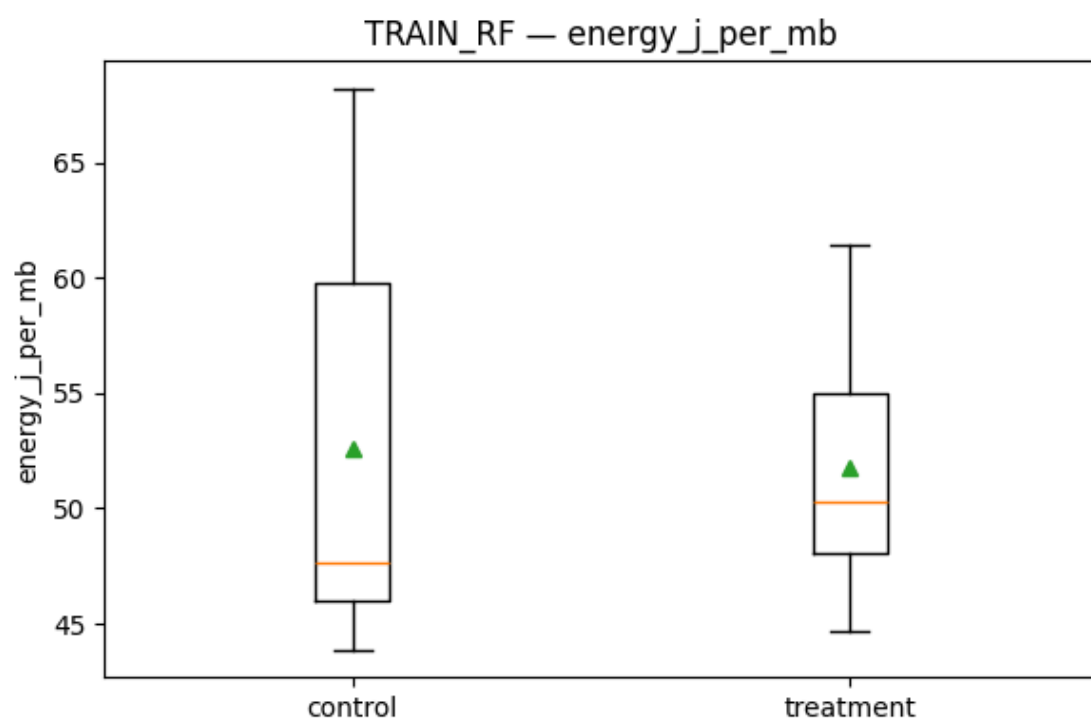
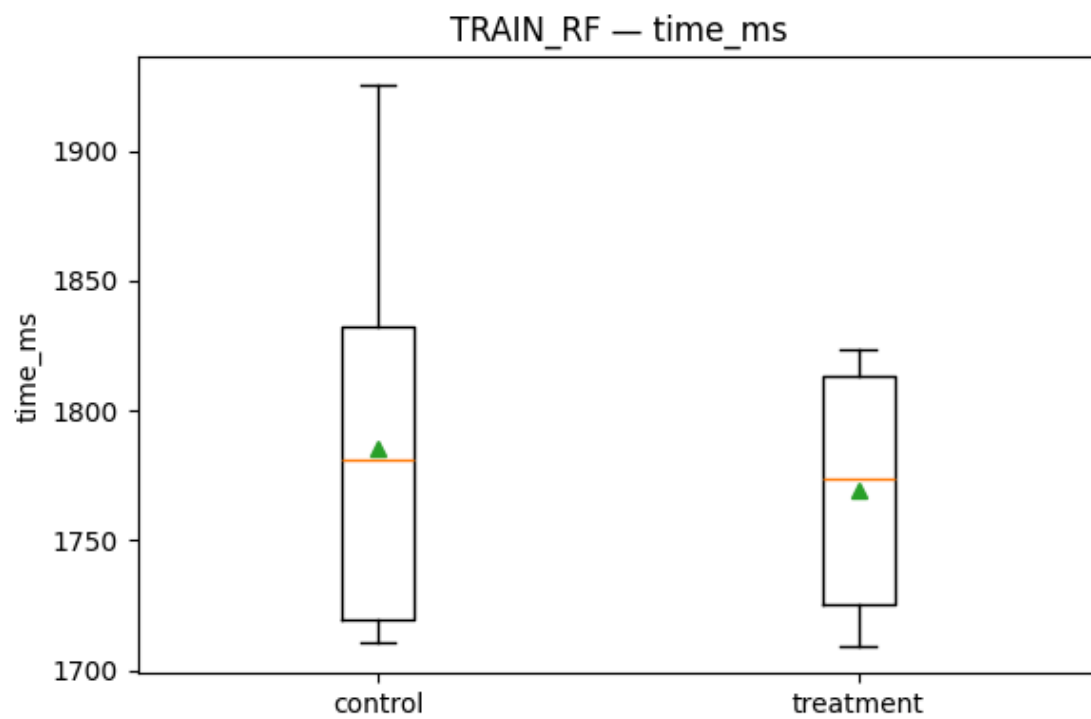
Datos, experimento y métricas

- Dataset sintético reproducible (seeds, metadatos).
- Escenarios: control vs. SMA; modelos: RF y SVM.
- Métricas: time_ms, cpu_avg, mem_peak_mb, records/s, energía: $J = W \times s$; normalizaciones J/MB, W/MB.

Fuente: `python-analysis/generate_dataset.py` , `python-analysis/aggregate_metrics.py` , `docs/energy_plan.md`

Resultados (visual 1)

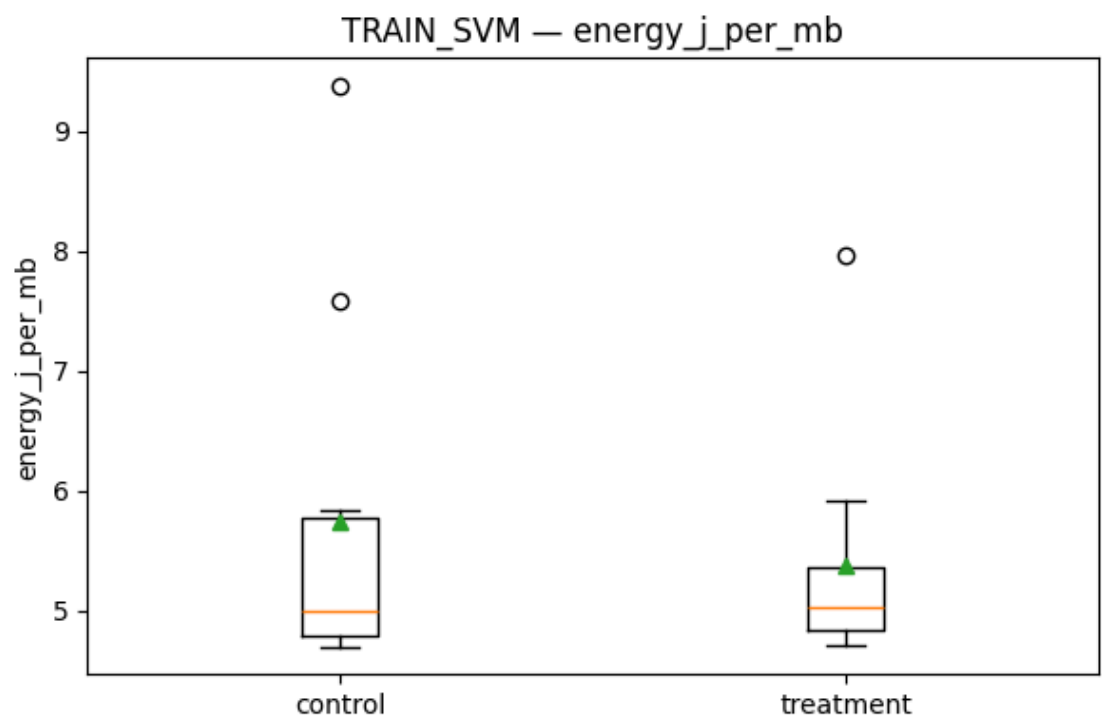
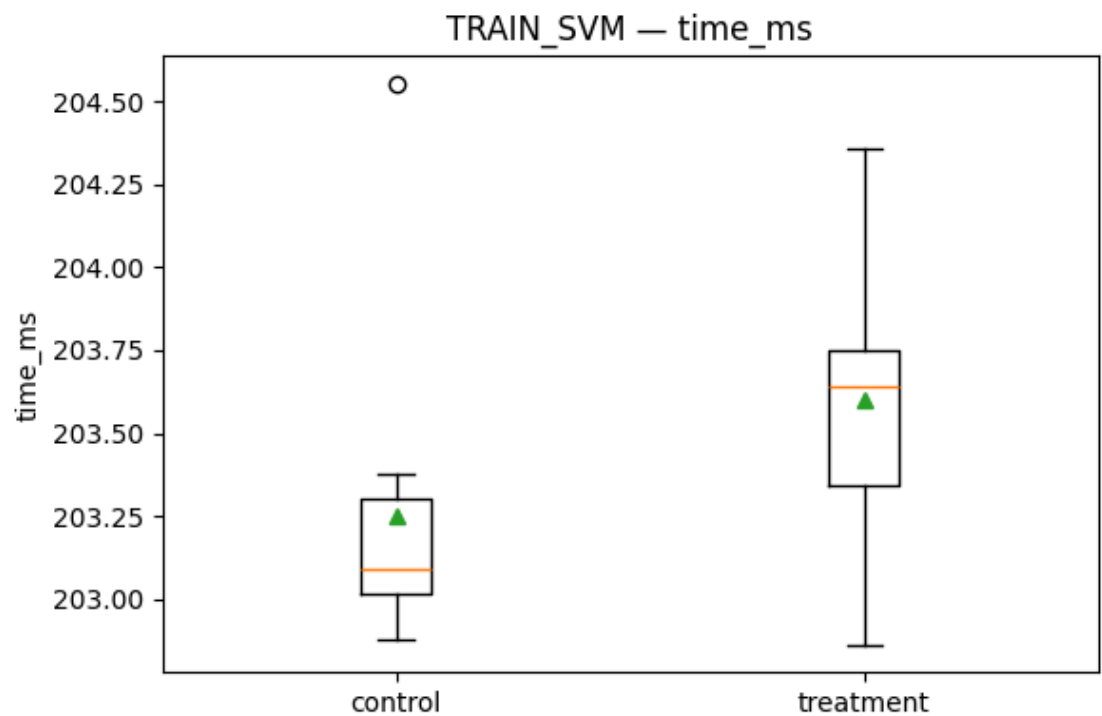
- TRAIN_RF — tiempo y energía por MB.



Fuente: docs/thesis_results_section.md (Sec. 7)

Resultados (visual 2)

- TRAIN_SVM — tiempo y energía por MB.



Fuente: docs/thesis_results_section.md (Sec. 7)

Análisis inferencial y validez

- Permutaciones (≥ 5000) por etapa y métrica; H_0 : sin diferencia entre grupos.
- Resultado: $p \geq 0.05$ en métricas principales (RF y SVM).
- Amenazas: proxy energético (TDP×CPU), dataset pequeño; mitigación: documentar y escalar.

Fuente: python-analysis/stats_analysis.py ,
data/results/stats_summary.{json,md}

Conclusiones principales

- SMA funcional y reproducible (monitoreo, reportes, scripts).
- Sin diferencias significativas en el escenario actual.
- El marco permite escalar condiciones y medir energía directa (RAPL/powercap) para reducir incertidumbre.

Fuente: docs/conclusions_implications_future_work.md

Aportes específicos

- Marco experimental reproducible (scripts, dashboards, normalizaciones J/MB).
- Arquitectura SMA y guía watts-first para energía.
- Capítulos de metodología, cronograma, resultados y checklist.

Fuente: docs/achievements/resultados_aporte.{md,tex}

Trabajo futuro y cierre

- Integrar medición directa (RAPL/powercap).
- Escalar datasets/concurrencia y ampliar modelos (XGBoost/LightGBM).
- Resiliencia/elasticidad: MTTR, reconfiguración, fallos inducidos.

Gracias. Preguntas.

Referencias cruzadas:

docs/thesis_results_section.md · docs/conclusions_implications_future_work.md ·
docs/appendix/reproducibility_checklist.md